Rebuilding Aquila's Auto-Resolve Battle Engine

A client brief from Ballista Games

IAT 461 | CLIENT PROPOSAL | JULY 21, 2026

1 Entity

Ballista Games is a small indie studio (around 40 people). We make *Aquila*, a historical real-time strategy game set in a made-up ancient world. Players build armies out of eight kinds of troops and fight big real-time battles on different terrain. Since a campaign has way more battles than anyone wants to play by hand, the game has an **auto-resolve** button that instantly decides who wins and how many troops each side loses. Lots of players use it, and lots of players complain about it. The usual complaint is that it gives them a result they would never get if they actually played the fight out, so they stop trusting the campaign. We want to fix it using our data instead of just guessing.

2 Business context and strategic options

Complaints have gone up since our last balance patch. Before we spend engineering time on this, we are choosing between three options:

- **Option A, remove auto-resolve** so every battle has to be played by hand.
- **Option B, add a flat rule** that gives the side with more troops a fixed bonus so the result feels about right.
- **Option C, rebuild the resolver from real battle data** so it actually looks at army makeup, terrain, and morale. On top of that, check whether our unit role labels still match how the units really play.

3 Problem definition

This breaks into an exploratory step and two modeling tasks. Work out the right method for each one from how it is described.

Validating the options (exploratory)

Look at the battle log and work out which of the three options makes sense. Two things to check. First, how often players actually use auto-resolve, which speaks to Option A. Second, how well a plain "bigger army wins" rule matches what really happened, and where it goes wrong, which speaks to Option B. We expect A and B to look bad, which is what points toward Option C.

Battle outcome model (tabular data)

If we go with Option C, predict which side wins a hand-played battle (the `outcome` column, attacker or defender) from the setup before the fight: how many of each troop type each side has, terrain, weather, average morale, general rating, attacker fatigue, whether the defender is dug in, and reinforcement timing. Just counting troops will not get you far, because both armies are usually about the same size and no single unit type is good on its own. What actually decides the fight is the matchups (like spears against cavalry, or cavalry against archers) and how the terrain helps or hurts them. You will probably need a round or two of feature engineering to pull that out, because it is not sitting there in the raw columns.

Unit role audit (text data)

Group the unit descriptions by what kind of fighter the text sounds like, without looking at the `advertised_role` tags, then compare your groups to those tags. We think our six marketing labels are out of date. Some of them probably mix together units that play very differently, and a few units are probably just labeled wrong. The point is to find that out from the description text on its own.

4 Stakeholder and audience

The main person using this is our **Lead Systems Designer** for the campaign, who owns the auto-resolve feature and reports to the Creative Director. The exploratory step tells them which option to pay for. The outcome model becomes the new auto-resolve, so it cannot be a total black box. The designer needs to see why it calls a battle the way it does so they can balance the game. The confidence per battle matters too, because we can auto-resolve the blowouts and give the close ones back to the player. The role audit goes to the content team, who will decide whether to split, merge, or relabel unit roles in the next patch.

5 Datasets

Everything is made-up data that I generated with a seeded random number generator (seed 461), so it is fully reproducible. I included the script, `generate_dataset.py`, so you can read it or regenerate everything. Three files are attached:

- `battles.csv`, 1,000 battles and 33 columns (for the exploratory step and the outcome model).
- `unit_roster.csv`, 240 units with text descriptions (for the role audit).
- `advertised_counters.graphml`, the in-game counter chart as a directed graph (6 role nodes, 6 edges). It is an optional helper for the outcome model and a reference for the role audit. Treat it as a marketing claim, not the truth.

Column dictionary, battles.csv

COLUMN	TYPE	DESCRIPTION
battle_id	string	Unique battle identifier.
attacker_faction / defender_faction	categorical	One of 8 factions; biases each side's composition.
terrain	categorical	plains, hills, forest, river_crossing, marsh.
weather	categorical	clear, rain, fog (affects ranged units most).
att_* (8 cols)	int	Attacker counts: heavy_infantry, spearmen, sword_infantry, archers, skirmishers, light_cavalry, shock_cavalry, artillery.
def_* (8 cols)	int	Defender counts, same eight types.
att_general_rating / def_general_rating	int 1 to 5	Commanding general skill.
att_avg_morale / def_avg_morale	int 40 to 100	Average starting morale.
att_fatigue	int 0 to 100	Attacker march fatigue; higher weakens the attacker.
def_fortified	0/1	Defender holds prepared positions.
att_reinforcement_minute	int	Minute attacker reinforcements arrive; 0 means none.
resolution_mode	categorical	auto or manual (how the battle was resolved in telemetry).
current_autoresolver_call	categorical	Winner the current (flawed) auto-resolver predicted.
att_casualty_pct / def_casualty_pct	float 0 to 1	Share of each side lost.
outcome	categorical	**Target.** True winner: attacker or defender.

Column dictionary, unit_roster.csv

COLUMN	TYPE	DESCRIPTION
unit_id	string	Unique unit identifier.
faction	categorical	Owning faction (8 total).
unit_name	string	In-game unit name (flavor).
advertised_role	categorical	Marketing tag: Line Infantry, Anti-Cavalry, Missile, Skirmisher, Cavalry, Siege. *Do not use it as a clustering input.*
description	text	Two to four sentences describing how the unit fights.

6 Success criteria

Validating the options. A clear, data-backed argument for why Options A and B do not work (how much people lean on auto-resolve, and how often a troop-count rule picks the wrong winner and in what kinds of battles), which leads to Option C.

The outcome model (prediction). Our current auto-resolver only matches the real winner about 63% of the time, so a good answer beats that by a solid margin. Just as important, it should be understandable enough that a designer can see which matchups and terrain drove a call and use that to balance the game. Good confidence scores help too, since we can auto-resolve the lopsided fights and hand the close ones back to the player.

The role audit (discovery). Groups that are specific enough to actually act on. Which advertised roles are lumping together units that play differently and should be split, and which single units look mislabeled. "The labels are basically fine" is an okay answer only if the groups clearly show that.

7 Constraints and notes

- **Random noise.** Outcomes are not fully deterministic, so the same setup will not always give the same winner. You cannot hit 100% and should not expect to.
- **Ground truth.** `current_autoresolver_call` is the old system's guess, not the real result. Only `outcome` is the truth. Think about whether auto and manual battles should be trained on the same way.
- **The counter graph is just a guess.** The GraphML is built from the same rough labels the role audit is checking, so it is not complete.
- **No sensitive data.** It is all synthetic game data, nothing personal.

Attachments: battles.csv, unit_roster.csv, generate_dataset.py, advertised_counters.graphml